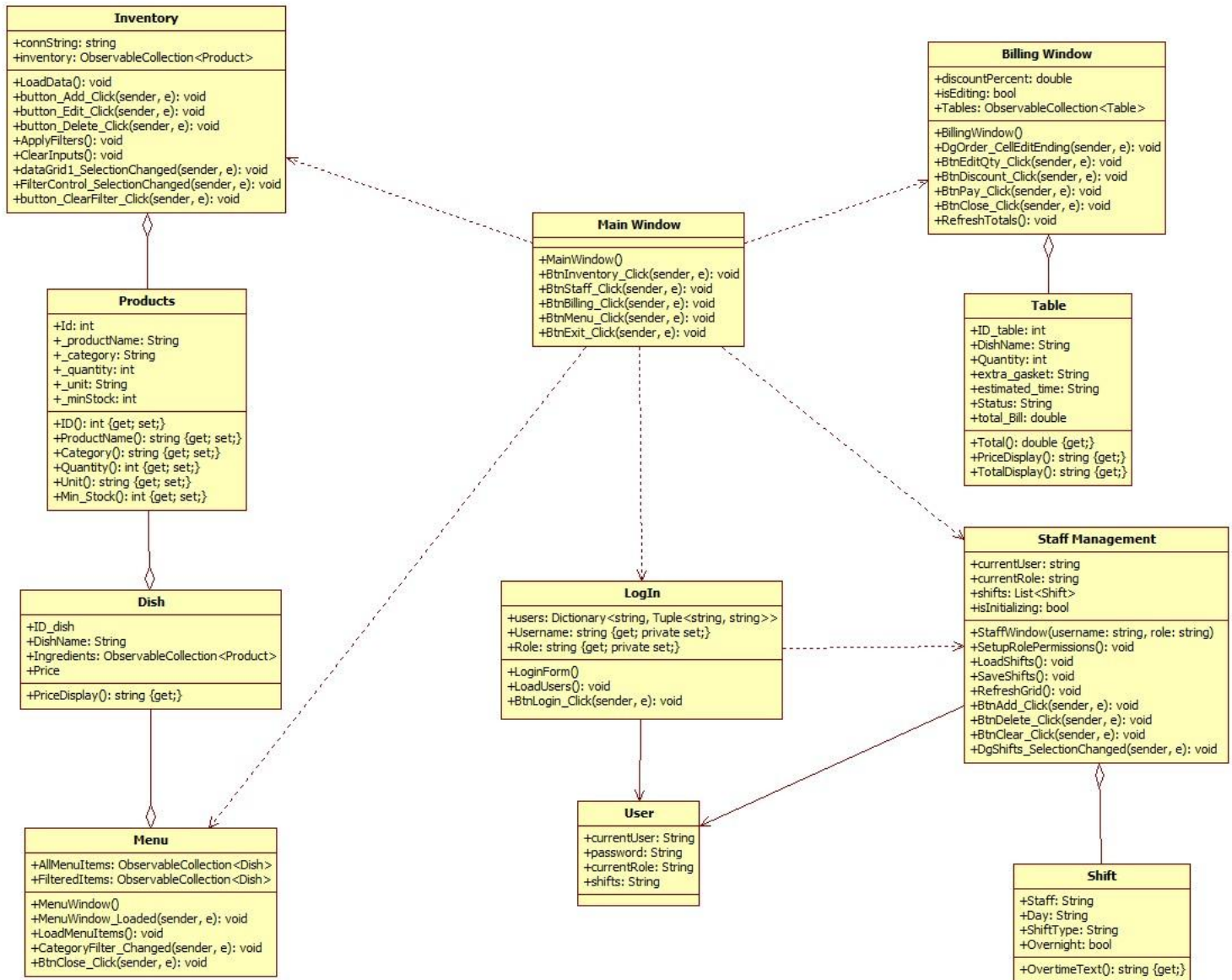


E2 – Software Modeling and Design Specifications

1. UML Class Diagram



2. Application Architecture Description

The **Bengos Restaurant Management System** is a multi-module system that utilizes a hybrid architectural approach. The system is designed with a clear separation of concerns, employing specific patterns (MVC, MVVM, and MVP) tailored to the different technologies used across its desktop and web components.

2.1. Module and Layer Structure

Each module in the project follows a classic three-layered architecture to ensure maintainability and scalability:

- **User Interface (UI) Layer:** This is the presentation layer. It consists of **WPF (XAML)** for the main dashboard and inventory, **WinForms** for staff and billing management, and **ASP.NET Core Razor Views** for the web-based customer menu.
- **Business Logic Layer:** This layer handles the core functionality, such as user authentication, role-based access control (Admin vs. Staff), discount calculations, and shift validation. These rules are encapsulated within **ViewModels, Presenters, or Controllers**.
- **Data Access Layer (DAL):** Manages how data is stored and retrieved. The system uses a variety of methods, including **Entity Framework Core 10.0** with SQLite, raw **ADO.NET** for SQL LocalDB, and **System.IO** for flat-text file persistence (users.txt, shifts.txt).

2.2. Architectural Specifications

The system implements the following architectural patterns to decouple the UI from the underlying business logic:

A. MVVM (Model-View-ViewModel)

Used in the **WPF** modules: **BengosRestaurantApp** and **inventoryWPF**.

- **Purpose:** To achieve a clean separation between the UI (View) and the presentation logic (ViewModel) via data binding and commands.
- **Implementation:** Logic is moved from the .xaml.cs code-behind files into ViewModels. The View binds to the ViewModel's properties and executes logic through ICommand objects rather than event handlers.

B. MVC (Model-View-Controller)

Used in the ASP.NET Core module: **DigitalClientMenu**.

- **Purpose:** To separate the application into three interconnected components: the Model (data), the View (HTML rendering), and the Controller (input handling).

- **Implementation:** The MenuController receives HTTP requests, interacts with the RestaurantContext (Model) to fetch data, and passes it to the Razor View for rendering.

C. MVP (Model-View-Presenter)

Used in the **WinForms** modules: **Staff_Management** and **Billing&Payments**.

- **Purpose:** Recommended for WinForms to make the View "passive," delegating all user actions to a Presenter.
- **Implementation:** The View (Form) implements an IView interface. The Presenter receives user events, processes the logic, and updates the View through the interface, making the logic fully unit-testable.

2.3. Architecture Summary Table

Module	Technology	Architecture Pattern	Data Access
Bengos RestaurantApp	WPF (.NET 8)	MVVM	Text files / In-memory
DigitalClientMenu	ASP.NET Core 10	MVC	EF Core + SQLite
Staff_Management	WinForms	MVP	Text files
Billing & Payments	WinForms	MVP	In-memory List
inventoryWPF	WPF (.NET 8)	MVVM	ADO.NET (SQL LocalDB)

3. User Interface Design

4. Description of the Main Classes

4.1. UI and Management Classes

These classes serve as the primary interface for users and manage the interaction between the user and the system logic.

Class Name	Role	Responsibilities
Main Window	Central Navigation Hub	Acts as the entry point of the application; manages navigation buttons to launch all other sub-modules.
LogIn	Security Manager	Handles user authentication; loads credentials from users.txt and assigns role-based permissions.
Inventory (Window)	Stock Controller	Manages the inventory interface; handles CRUD operations for products and applies category filters.
Staff Management	Human Resources Manager	Facilitates shift scheduling; implements role-based access for Admin and Staff users.
Billing Window	Order Processor	Manages the current order summary; allows quantity edits and calculates subtotals.
Menu (Window)	Product Browser	Provides a visual interface for browsing restaurant dishes and their specific ingredients.

4.2. Data Model Entities

These classes define the structure of the data objects used throughout the application layers.

Class Name	Role	Responsibilities
Products	Stock Entity	Stores specific product data including Name, Category, Quantity, Unit, and Minimum Stock.
Dish	Menu Entity	Represents a menu item; contains a collection of ingredients (Products) and pricing data.
User	Identity Entity	Stores security credentials (username/password) and the specific system role.
Shift	Schedule Entity	Represents a work interval; tracks staff name, day, shift type, and overtime status.
Table (Order Item)	Transactional Entity	Tracks individual items within an order, including dish names, quantities, and total bill calculations.